Random Forest Algorithm in Machine Learning

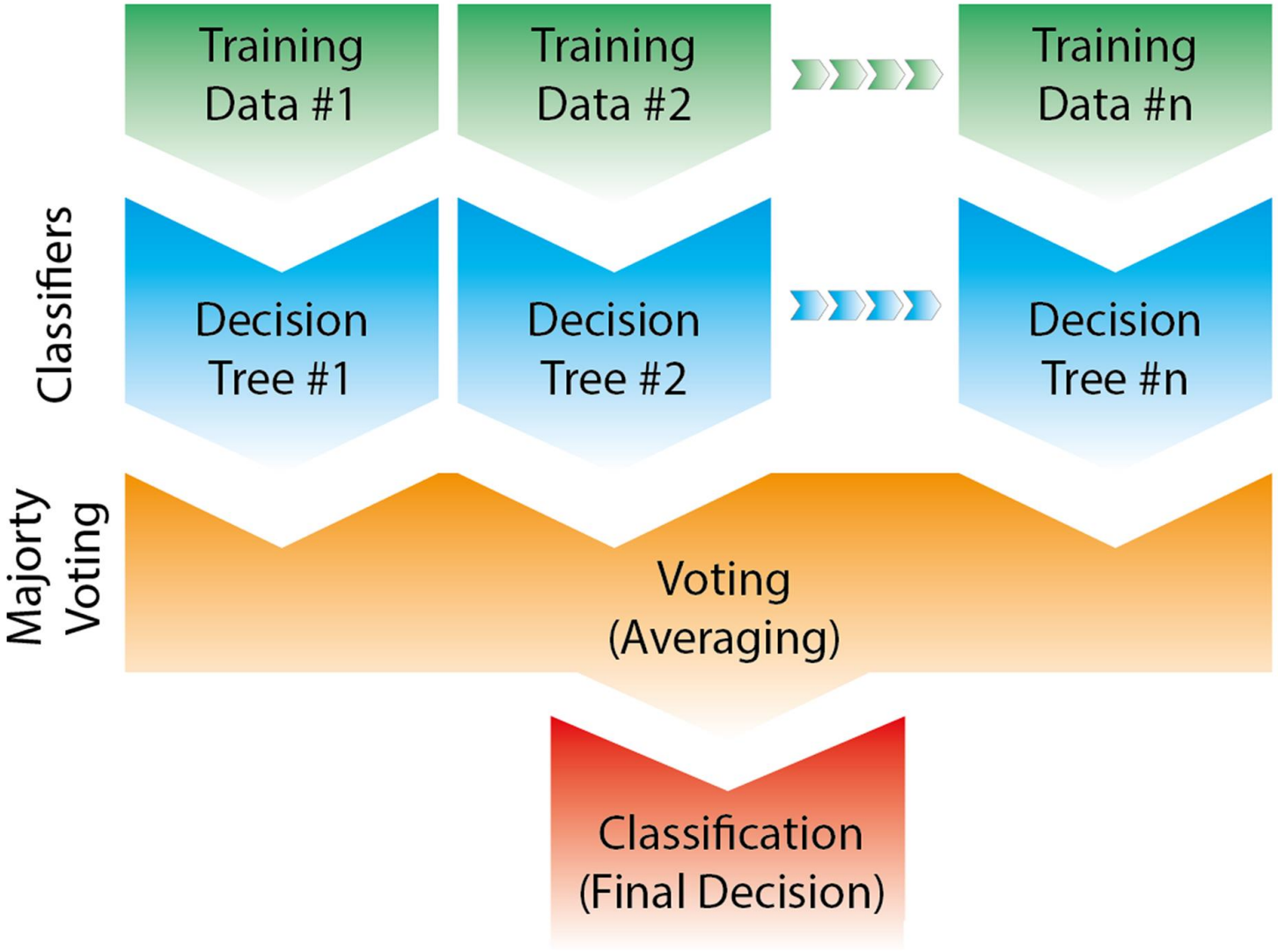
- Santhosh Kumar

Introduction:

- Random forest is a machine learning algorithm based on **ensemble learning**.
- Combines multiple **decision trees** to make a more accurate and stable prediction.
- It can be used for both **classification** and **regression** tasks.

Why is it called a "Forest"?

- Each **decision tree** is like a "tree."collection (ensemble) of many trees forms a **"forest."**
- Instead of relying on one tree, the model takes a **majority vote** from many trees → leading to more reliable predictions.



Decision Tree:

- A **decision tree** splits data into branches based on feature conditions.
- **Strengths:** simple, interpretable, handles mixed data types.
- **Limitation:**

  1.prone to **overfitting** → memorizes training data instead of generalizing.

  2. Decision Trees has high variance(it is very sensitive to small changes in the training data)

Ensembling:

- Ensembling is a **machine learning technique** where we combine predictions from **multiple models (learners)** to build a stronger overall model.
- A single model may suffer from **bias** (underfitting) or **variance** (overfitting).
- By combining models, we balance out individual weaknesses and achieve **better accuracy, stability, and generalization**.

Ensembling Methods in Random Forest :

1. Bagging:

- Train multiple base learners (decision trees) on different **bootstrap samples** of the dataset.
- Combine predictions by **majority vote** (classification) or **average** (regression).
- Reduces **variance** and improves stability.

2. Feature Randomness (unique to Random Forest):

- At each split, Random Forest randomly selects a **subset of features**.

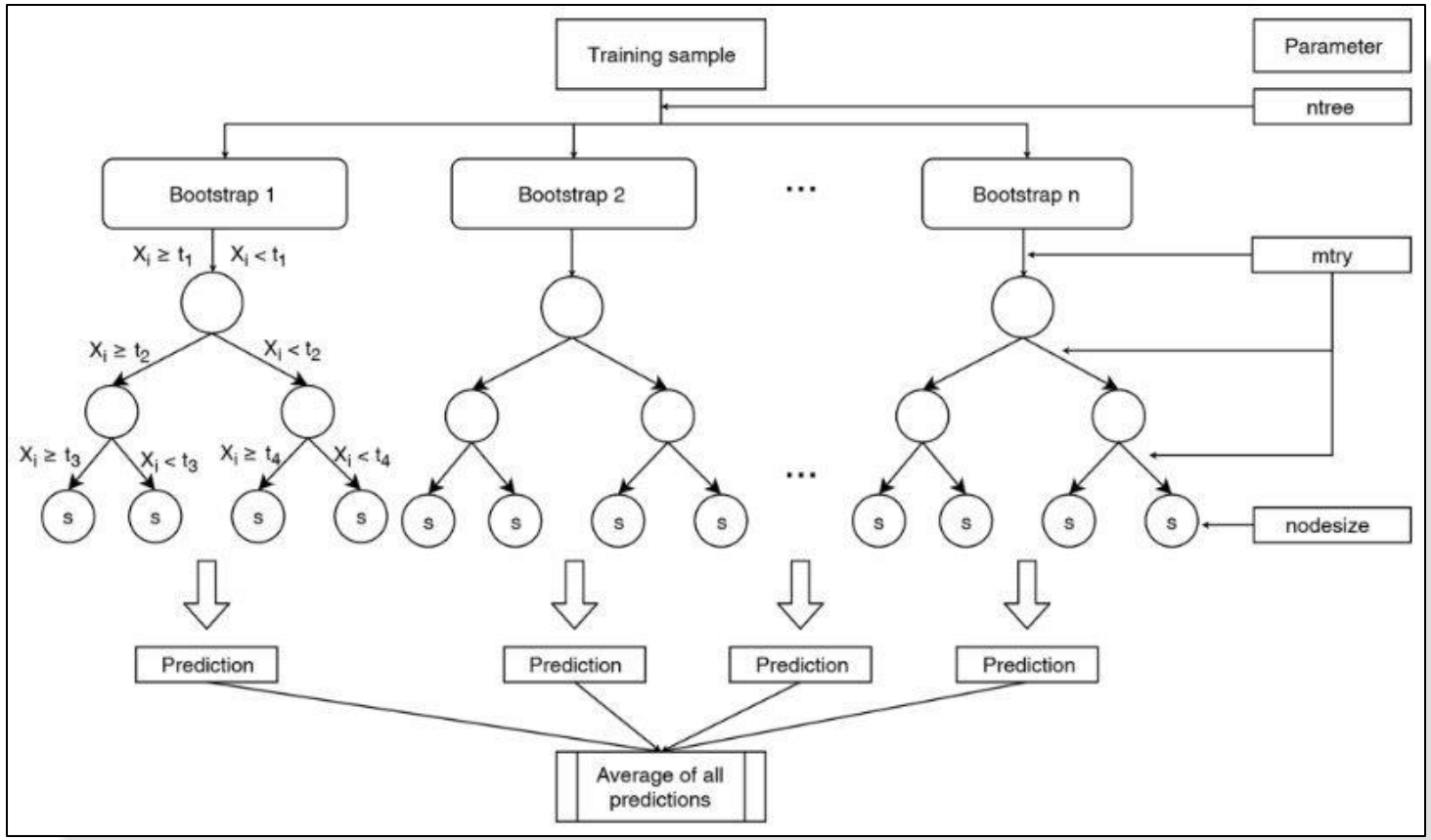
Ensembling Methods in Random Forest :

- Prevents dominant features from being chosen every time.
- Creates more diverse trees and reduces correlation among them.

- **3. Out-of-Bag (OOB) Estimation:**
- Since each tree trains on ~63% of the dataset (bootstrap sample), the remaining ~37% (not sampled) is used as a **validation set**.
- Provides an **internal performance estimate** without needing extra cross-validation.

How Random Forest Classifier Works:

1. Dataset
2. Bootstrap samples
3. Trees
4. Voting



How Random Forest Classifier Works:

1. Bootstrap Sampling:

- From the original dataset, create multiple **random samples with replacement**.
- Each sample is used to train a different decision tree.
- One Bootstrap sample is used to grow one decision tree.

2. Train Multiple Trees

- Each tree is trained on its own bootstrap sample using random feature splits.

How Random Forest Classifier Works:

- Trees are usually deep (not pruned) to capture complex patterns.

3. Voting (Ensembling)

- For **classification**: Each tree votes for a class → final prediction is by **majority vote**.
- For **regression**: Predictions are averaged across trees.

Syntax and parameters(Python / scikit-learn): Classification

- ***class* sklearn.ensemble.RandomForestClassifier(*n_estimators*=*100*, ***, *criterion*=*'gini'*, *max_depth*=*None*, *min_samples_split*=*2*, *min_samples_leaf*=*1*, *min_weight_fraction_leaf*=*0.0*, *max_features*=*'sqrt'*,*max_leaf_nodes*=*None*, *min_impurity_decrease*=*0.0*, *bootstrap*=*True*, *oob_score*=*False*, *n_jobs*=*None*, *random_state*=*None*, *verbose*=*0*, *warm_start*=*False*, *class_weight*=*None*, *ccp_alpha*=*0.0*,*max_samples*=*None*, *monotonic_cst*=*None*)

Syntax and parameters(Python / scikit-learn): Regression

- ***class* sklearn.ensemble.RandomForestRegressor(*n_estimators*=*100*, *, *criterion*='*squared_error*', *max_depth*=*None*, *min_samples_split*=*2*, *min_samples_leaf*=*1*, *min_weight_fraction_leaf*=*0.0*, *max_features*=*1.0*, *max_leaf_nodes*=*None*, *min_impurity_decrease*=*0.0*, *bootstrap*=*True*, *oob_score*=*False*, *n_jobs*=*None*, *random_state*=*None*, *verbose*=*0*, *warm_start*=*False*, *ccp_alpha*=*0.0*, *max_samples*=*None*, *monotonic_cst*=*None*)

Important Parameters:

- **n_estimators:** Number of trees in the forest
- **criterion:** Function to measure split quality (gini or entropy)
- **max_depth:** Maximum depth of each tree
- **min_samples_split:** Minimum samples required to split a node
- **min_samples_leaf :** Minimum samples required at a leaf node
- **max_features:** Number of features to consider for a split
- **random_state:** Ensures reproducible results

Advantages:

- Handles high-dimensional data.
- Reduces overfitting (compared to single decision trees).
- Robust to noise and missing values.
- Works well with categorical + numerical data.
- Provides feature importance.
- High accuracy and robustness
- Less prone to hyperparameter tuning

Limitations:

- Computationally heavy with very large datasets.
- Less interpretable compared to a single decision tree.
- Requires tuning (can be slower for real-time predictions).

Applications:

- **Finance and banking:** Credit risk assessment, Fraud detection, Algorithmic trading
- **Healthcare:** Disease diagnosis and risk prediction, Drug discovery, Medical image analysis
- **E-commerce and retail:** Recommendation engines, Price optimization, Customer behavior prediction
- **Image and text classification:** Image recognition, Spam detection

Links:

- RandomForest_Regressor
- RandomForest_Classifier
- Ensembling
- YouTube

Thank you!!!

☮